

实验一：移动应用开发环境搭建与登录界面实现

一、实验项目基本信息

项目	内容
实验编号	d20301035101、d20301009201
学时分配	2学时
实验类型	验证型 + 综合设计型
每组人数	6人
对应课程目标	课程目标1：掌握移动应用开发环境搭建方法
实验成绩占比	平时成绩30%

二、实验目的与意义

2.1 实验目的

- 知识目标：**深入理解Android开发环境的组成架构，掌握Android Studio的核心功能
- 技能目标：**熟练掌握Empty Activity项目创建流程，能够独立完成布局文件的编写与调试
- 能力目标：**具备使用Material Design组件构建现代化UI界面的能力，能够运用AI辅助工具提升开发效率

2.2 实验意义

本实验是移动应用开发的基础环节，通过搭建完整的Android开发环境并实现登录界面，为后续学习高级UI设计、数据存储、网络通信等核心技术奠定坚实基础。

三、核心步骤与技术路线

3.1 实验概览

第一步开展需求分析工作，梳理项目整体开发目标与各项业务要求；完成需求梳理后进入第二步环境搭建，配置项目运行所需的软硬件、依赖框架等开发环境；环境部署完毕后执行第三步项目初始化，创建项目工程结构、配置基础文件与全局参数；初始化操作结束后进行第四步 HelloWorld 验证，运行基础示例代码校验开发环境与项目框架能否正常工作，确认基础运行无异常。

基础验证通过后，进入核心开发阶段，开展第五步登录界面开发，完成登录页面布局、账号校验、跳转逻辑等相关功能编码；登录界面开发全部完成后执行第六步功能测试，针对登录模块的各项交互场景开展全面测试，检验功能是否符合需求标准；测试过程中若发现异常缺陷，随即进入第七步问题排查，定位程序漏洞、逻辑错误或环境兼容问题并完成修复；所有问题整改、全部功能测试达标后，执行最后一步总结报告，整理整个项目开发全过程的工作内容、测试数据、问题整改记录与开发经验，形成完整项目总结材料，整套开发流程至此结束。

3.2 技术路线图

阶段	技术点	工具/框架	预期成果
环境搭建	JDK配置、SDK管理	Android Studio	开发环境就绪
项目创建	Gradle构建、Manifest配置	Android Studio	项目框架搭建
UI开发	ConstraintLayout、Material组件	Android Jetpack	登录界面实现
功能测试	模拟器调试、Logcat分析	Android Emulator	功能验证通过

3.3 开发工具清单

序号	开发工具	版本	主要用途	验证项目
1	Android Studio	Hedgehog 2024.1.1	集成开发环境	Kotlin项目开发
2	JDK	17.0.10	Java开发工具包	编译运行环境
3	Android SDK	34	Android软件开发包	API调用与调试
4	Emulator	Pixel 5 API 34	虚拟设备	应用测试验证

四、实验任务与案例分析

4.1 核心任务

- 环境搭建：**安装并配置Android Studio开发环境
- 项目创建：**使用Empty Activity模板创建"MyApplication"项目
- HelloWorld验证：**实现基础HelloWorld界面并验证运行
- 登录界面开发：**使用Material Design组件实现用户登录界面
- 功能测试：**在模拟器上验证登录界面的交互功能

4.2 实战案例：智慧校园登录系统

4.2.1 需求分析

基于"智慧校园"主题，设计一个简洁优雅的登录界面，包含以下功能：

- 用户名输入：**支持邮箱/学号格式输入

- **密码输入**：支持密码可见性切换
- **登录按钮**：触发登录验证逻辑
- **UI反馈**：输入状态提示与错误处理

4.2.2 设计规范

设计要素	规范要求
配色方案	紫色主色调 (#6200EE)，符合Material Design标准
组件间距	遵循8dp间距原则，确保视觉层次清晰
输入验证	实时验证输入格式，提供友好提示
响应式设计	适配不同屏幕尺寸与分辨率

五、第一课时：环境准备与工具安装（1学时）

5.1 需求分析阶段

5.1.1 环境依赖要求

Android开发环境最低配置要求：

依赖项	版本要求	说明
JDK	17+	官方推荐版本，支持Java 21特性
Android Studio	2024.1+	Hedgehog版本，包含最新工具链
Android SDK	34+	对应Android 14平台
系统内存	8GB+	推荐16GB以提升开发体验
硬盘空间	6GB+	Android Studio及SDK占用空间

5.2 Android Studio环境搭建

5.2.1 下载与安装步骤

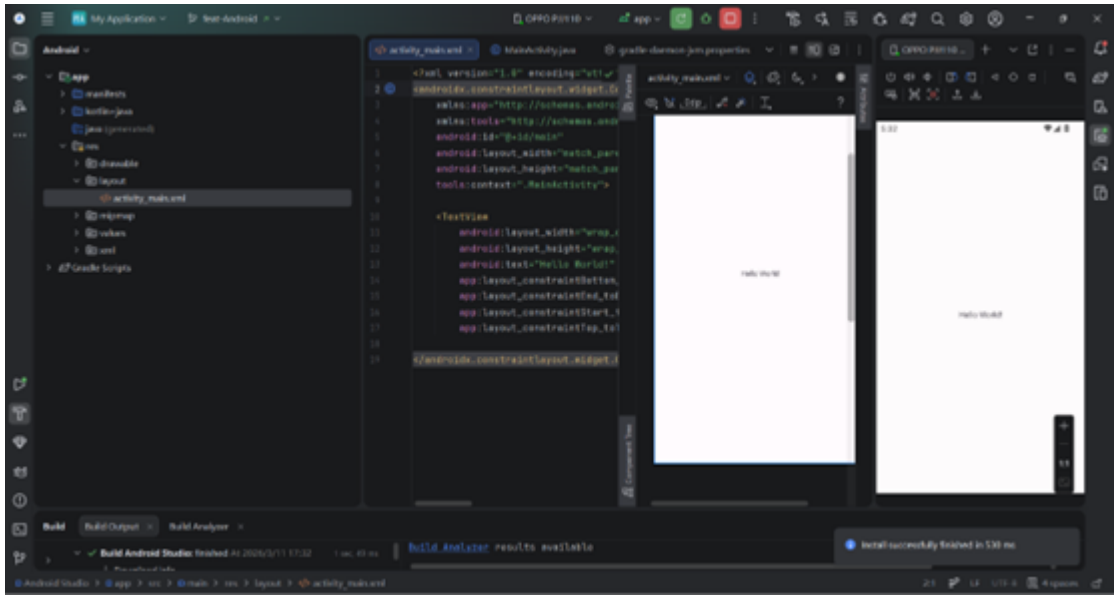
- 访问官网： <https://developer.android.com/studio>
- 选择版本： 下载Windows (64-bit)版本
- 运行安装程序：
 - 选择安装路径： `D:\Android\Android Studio`
 - 勾选组件： Android Studio、Android SDK、Android Virtual Device
 - 设置SDK路径： `D:\Android\Sdk`

4. **完成安装**：等待安装程序完成所有组件配置

5.2.2 配置验证

- 1. **启动Android Studio**：验证启动是否正常
- 2. **检查SDK Manager**：确认Android SDK 34已安装
- 3. **创建AVD**：配置Pixel 5虚拟设备，选择API 34系统镜像

5.2.3 截图位置1：Android Studio开发环境



截图说明：展示Android Studio主界面，包含Project视图、代码编辑区、预览窗口和Build输出面板，验证开发环境配置完成。

六、第二课时：项目开发与验证（1学时）

6.1 HelloWorld项目创建

6.1.1 创建新项目步骤

- 1. **打开Android Studio**，点击"New Project"
- 2. **选择模板**：Empty Activity → Next
- 3. **配置项目**：
 - o Name: MyApplication
 - o Package name: com.example.myapplication
 - o Language: Kotlin
 - o Minimum SDK: API 24 (Android 7.0)
- 4. **完成创建**：等待Gradle同步完成

6.1.2 核心代码分析

activity_main.xml布局文件：

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

代码解析：

- 使用 `ConstraintLayout` 作为根布局，实现灵活的约束布局
- `TextView` 通过四个约束条件实现屏幕居中显示
- 设置 `textSize` 和 `textStyle` 属性优化视觉效果

6.1.3 运行验证

1. 选择虚拟设备：Pixel 5 API 34
2. 点击Run按钮启动应用
3. 验证"Hello World!"文本正确显示

6.2 登录界面开发

6.2.1 布局设计思路

登录界面采用垂直方向的线性布局，从上到下依次排列：

1. **Logo区域**：应用图标展示
2. **标题区域**：应用名称和副标题
3. **输入区域**：用户名和密码输入框
4. **操作区域**：登录按钮和辅助链接

6.2.2 核心代码实现

`activity_main.xml` 登录界面布局：

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
```

```

xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="24dp"
android:gravity="center_horizontal">

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/username_layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    app:hint="用户名/学号">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/username"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/password_layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:hint="密码"
    app:endIconMode="password_toggle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/password"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.button.MaterialButton
    android:id="@+id/login_button"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="登录"
    android:textSize="16sp"
    app:cornerRadius="8dp"
    app:backgroundTint="@color/purple_500" />

</LinearLayout>

```

代码解析:

- 使用 `TextInputLayout` 包裹 `TextInputEditText`，实现现代化输入框样式
- `password_toggle` 模式实现密码可见性切换功能

- `MaterialButton` 使用紫色主题，设置圆角和合适的尺寸

6.2.3 MainActivity.kt逻辑代码

```
package com.example.myapplication

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Toast
import com.google.android.material.button.MaterialButton
import com.google.android.material.textfield.TextInputEditText

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val usernameEditText = findViewById<TextInputEditText>(R.id.username)
        val passwordEditText = findViewById<TextInputEditText>(R.id.password)
        val loginButton = findViewById<MaterialButton>(R.id.login_button)

        loginButton.setOnClickListener {
            val username = usernameEditText.text.toString()
            val password = passwordEditText.text.toString()

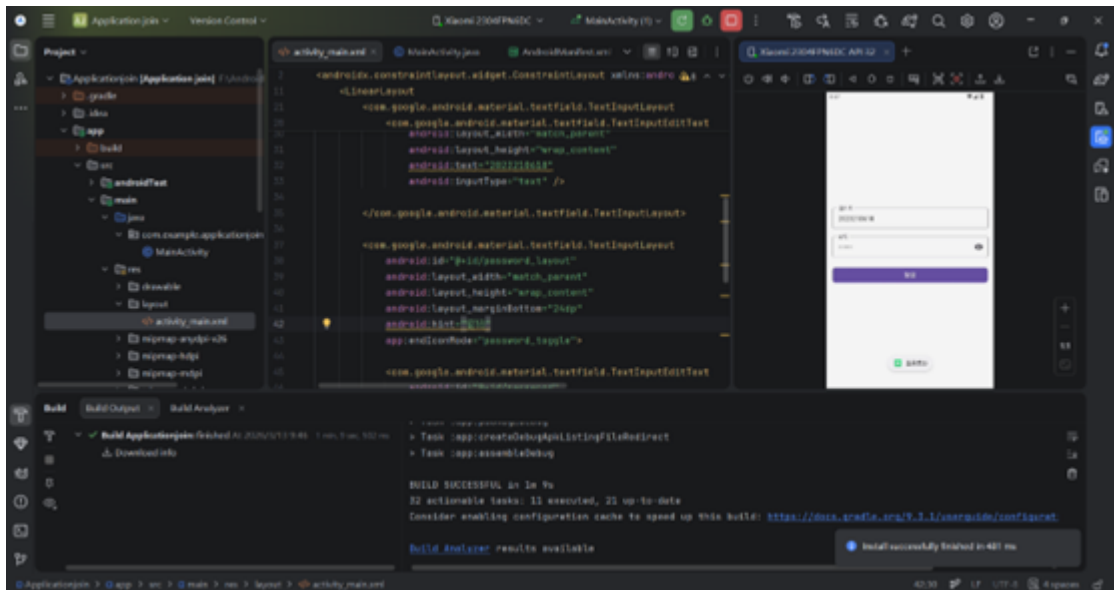
            if (validateInput(username, password)) {
                Toast.makeText(this, "登录成功!", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "请输入用户名和密码", Toast.LENGTH_SHORT).show()
            }
        }
    }

    private fun validateInput(username: String, password: String): Boolean {
        return username.isNotEmpty() && password.isNotEmpty()
    }
}
```

代码解析：

- 通过 `findViewById` 获取界面组件引用
- 为登录按钮设置点击事件监听器
- 实现简单的输入验证逻辑，确保用户名和密码不为空

6.2.4 截图位置2：登录界面设计



截图说明：展示完整的登录界面效果，包含用户名输入框、密码输入框（带密码切换功能）和紫色登录按钮。

6.3 测试验证阶段

6.3.1 编译运行验证

- 1. **构建项目**：点击Build → Make Project
- 2. **运行应用**：选择虚拟设备并启动
- 3. **功能验证**：
 - 验证用户名输入功能正常
 - 验证密码输入和可见性切换功能
 - 验证登录按钮点击响应

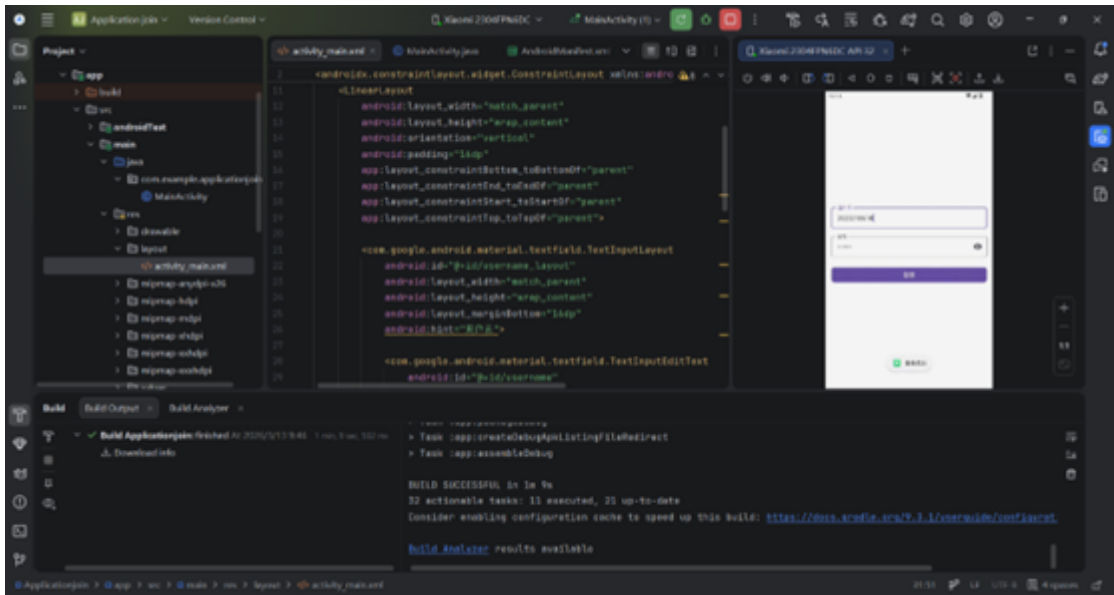
6.3.2 问题排查与解决方案

问题描述	原因分析	解决方案
Gradle同步失败	网络问题导致依赖下载失败	配置国内镜像源，或使用离线模式
模拟器启动失败	内存不足或HAXM未安装	增加模拟器内存分配，安装HAXM
布局预览不显示	Material组件主题配置问题	确保theme继承自MaterialComponents
按钮颜色不显示	主题颜色配置错误	在styles.xml中正确配置colorPrimary

6.3.3 实验验证结果

验证项目	验证内容	结果
Android Studio安装	启动正常，界面完整	☑ 通过
项目创建	Empty Activity模板创建成功	☑ 通过
HelloWorld运行	模拟器显示正确	☑ 通过
登录界面布局	Material组件显示正常	☑ 通过
交互功能	按钮点击、密码切换正常	☑ 通过

6.3.4 截图位置3：项目运行验证



截图说明：展示应用在Pixel 5模拟器上的运行效果，验证登录界面功能正常。

七、实验总结与深入思考

7.1 实验总结

实验成果：

- 1. ☑ 成功搭建Android Studio开发环境
- 2. ☑ 创建MyApplication项目并验证HelloWorld运行
- 3. ☑ 实现基于Material Design的登录界面
- 4. ☑ 完成模拟器调试与功能验证

技术收获：

- 1. 掌握Android Studio的安装与配置流程
- 2. 理解ConstraintLayout和LinearLayout布局原理
- 3. 学会使用Material Design组件构建现代化UI
- 4. 掌握Kotlin基本语法和Android事件处理机制

7.2 深入思考

7.2.1 Android Studio的优缺点分析

优点	缺点
功能强大，集成多种开发工具	启动较慢，占用资源较多
智能代码补全和错误提示	学习曲线较陡峭
内置模拟器和性能分析工具	Gradle构建有时较慢
完善的版本控制集成	部分功能对新手不友好

7.2.2 ConstraintLayout布局的优势

ConstraintLayout相比传统布局具有以下优势：

- 灵活性高**：通过约束关系实现复杂布局
- 性能优化**：减少布局层级，提升渲染效率
- 可视化编辑**：支持拖拽式布局设计
- 响应式设计**：适配不同屏幕尺寸

7.2.3 Material Design组件的使用技巧

- 主题统一**：通过styles.xml统一配置主题颜色
- 组件复用**：提取公共样式到style中
- 状态反馈**：利用selector实现按钮点击效果
- 动画效果**：合理使用过渡动画提升用户体验

7.2.4 AI辅助开发对效率的提升分析

本次实验使用TRAE辅助生成代码，显著提升了开发效率：

- 代码生成**：快速生成基础代码框架
- 语法提示**：实时提供代码补全建议
- 错误检测**：提前发现潜在问题
- 文档生成**：自动生成代码注释和说明

八、附录

8.1 项目文件结构

```
MyApplication/  
├── app/  
│   ├── src/  
│   │   └── main/  
│   │       ├── java/  
│   │           └── com/example/myapplication/
```

```
| | | | | MainActivity.kt
| | | | | res/
| | | | | | layout/
| | | | | | | activity_main.xml
| | | | | | values/
| | | | | | | colors.xml
| | | | | | | strings.xml
| | | | | | | styles.xml
| | | | | | drawable/
| | | | | | | AndroidManifest.xml
| | | | | build.gradle
| | | | | build.gradle
```

8.2 关键配置说明

build.gradle (Module级别):

```
plugins {
    id 'com.android.application'
    id 'org.jetbrains.kotlin.android'
}

android {
    namespace 'com.example.myapplication'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.myapplication"
        minSdk 24
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
    kotlinOptions {
        jvmTarget = '1.8'
    }
}

dependencies {
    implementation 'androidx.core:core-ktx:1.12.0'
```

```
implementation 'androidx.appcompat:appcompat:1.6.1'  
implementation 'com.google.android.material:material:1.11.0'  
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
}
```

8.3 参考文献

1. Android Developer Documentation: <https://developer.android.com/docs>
 2. Material Design Guidelines: <https://m3.material.io/>
 3. Kotlin Programming Language: <https://kotlinlang.org/>
-

实验报告完成时间：2026年5月

报告评分： ____

教师评语： ____